Task 2

ASP.NET Core Web & MVC – Concept Questions

#	Question
1	Why we use IActionResult, not ActionResult
2	What an HttpContext request and response message consist of
3	Difference between HTTP and HTTPS
4	Segments and fragments in a URL (real example)
5	Builder and Dependency Injection (real-life example)
6	Razor Pages vs MVC, two business cases
7	Content-Type in the response message
8	Minification, bundling, Webpack, lazy loading

Q1. Why do we use IActionResult not ActionResult? (with scenario / problems)

ActionResult is an abstract *class*; **IActionResult** is an *interface* that every result type (ViewResult, JsonResult, RedirectResult, NotFoundResult, ...) implements. Declaring the return type as the interface means "this action can return *any* kind of result", and the action does not depend on one specific base class.

Scenario

An action that shows a product page: if the product exists return a View, if the id is invalid return BadRequest, if it does not exist return NotFound, and for an AJAX request return JSON.

```
public IActionResult Details(int id, bool ajax = false)
{
    if (id <= 0)  return BadRequest();          // 400
    var p = _repo.Get(id);
    if (p == null) return NotFound();           // 404
    if (ajax)     return Json(p);               // JSON
    return View(p);                             // HTML view
}
```

Problems when we do not use it

- **Specific return types break flexibility:** if the method is declared `ViewResult`, `return NotFound();` does not compile because NotFoundResult is not a ViewResult.
- **Tight coupling:** a method typed with the class ActionResult can only return types that inherit from that class. A custom result (e.g. `PdfResult`) that only implements the interface cannot be returned.
- **Unit testing:** with the interface we simply test the outcome: `Assert.IsType<NotFoundResult>(result)`; the contract stays open for mocks and custom results.
- **Convention / maintainability:** MVC and Web API templates use IActionResult, so the action can later change its response without changing its signature.

Note: in Web APIs, `ActionResult<T>` is often preferred, because it keeps the flexibility *and* exposes the return type T to Swagger/OpenAPI.

Q2. What do the HttpContext request and response messages consist of?

HttpContext wraps everything about one HTTP call: `Request`, `Response`, `User`, `Session`, `Items`, `Connection` and `RequestServices`.

Part	Request message (client to server)	Response message (server to client)
1. Start line	Request line: **method** (GET/POST/PUT/DELETE), **path + query**, **HTTP version**	Status line: **HTTP version**, **status code** (200, 404, 500), **reason phrase**
2. Headers	Host, User-Agent, Accept, Content-Type, Content-Length, Cookie, Authorization	Content-Type, Content-Length, Set-Cookie, Cache-Control, Location, Server
3. Empty line	Separates headers from body	Separates headers from body
4. Body (optional)	Form data, JSON, uploaded file (POST/PUT)	HTML page, JSON, image, file

Part	Request message (client to server)	Response message (server to client)
In ASP.NET Core	`HttpContext.Request`: Method, Path, QueryString, Headers, Cookies, Form, Body, Scheme, Host	`HttpContext.Response`: StatusCode, Headers, Cookies, ContentType, Body, Redirect()

Raw request example

```
POST /Account/Login HTTP/1.1
Host: www.example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 31
Cookie: session=abc123

email=ali@mail.com&password=****
```

Raw response example

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 1523
Set-Cookie: auth=xyz; HttpOnly

<html> ... </html>
```

Q3. What is the difference between HTTP and HTTPS?

Aspect	HTTP	HTTPS
Full name	HyperText Transfer Protocol	HTTP Secure (HTTP over TLS/SSL)
Default port	80	443
Encryption	None, data travels as plain text	Encrypted with TLS
Integrity	Data can be altered in transit	Tampering is detected
Authentication	Server identity is not verified	Server proven by an SSL/TLS certificate
Risks	Sniffing and man-in-the-middle attacks (passwords, cards visible)	Protected against both
Browser	Shows "Not secure" warning	Padlock icon; required by many modern browser features
SEO / speed	No ranking benefit; HTTP/2 usually needs HTTPS	Ranking signal; enables HTTP/2 and HTTP/3
URL	http://example.com	https://example.com

In ASP.NET Core we enable it with:

```
app.UseHttpsRedirection();   // redirect http -> https
app.UseHsts();               // tell browsers to always use https
```

Q4. What are segments and fragments in a URL? (real URL example)

```
https://en.wikipedia.org/wiki/Hypertext_Transfer_Protocol?action=raw#History

scheme   : https
host     : en.wikipedia.org
segment 1: wiki
segment 2: Hypertext_Transfer_Protocol
query    : ?action=raw
fragment : #History
```

Part	Value in the example	Meaning
Scheme	https	Protocol used
Host	en.wikipedia.org	Server / domain
Segments	**wiki** and **Hypertext_Transfer_Protocol**	The parts of the path separated by `/`; used by the server/router to find the resource
Query string	?action=raw	key=value pairs sent to the server
Fragment	**#History**	Points to a section inside the page; handled by the **browser only**, it is never sent to the server

MVC example

```
URL:      https://shop.com/Products/Details/15
Pattern:  {controller=Home}/{action=Index}/{id?}
Segments: Products -> controller | Details -> action | 15 -> id
          so it runs ProductsController.Details(15)
```

Q5. What are the Builder and Dependency Injection? (real-life example)

A) Builder

Builder is a design pattern that constructs a complex object **step by step**, then produces the final object with a `Build()` call. **Real life:** ordering a custom sandwich: choose the bread, add the meat, add vegetables, add sauce, and only at the end the cashier "builds" and hands you the sandwich. Different choices give different sandwiches with the same process.

In ASP.NET Core, `WebApplication.CreateBuilder(args)` returns a builder where we configure services and settings, then `builder.Build()` creates the app.

```
var builder = WebApplication.CreateBuilder(args);   // 1. start the builder
builder.Services.AddControllersWithViews();         // 2. configure services
builder.Services.AddScoped<IOrderService, OrderService>();
var app = builder.Build();                          // 3. build the app
app.MapDefaultControllerRoute();
app.Run();
```

B) Dependency Injection (DI)

DI means a class does not create the objects it depends on; they are **given (injected)** to it, normally through the constructor, by a DI container. This gives loose coupling, easy testing and easy swapping of implementations. **Real life:** a restaurant chef does not go shopping for ingredients or build the oven; a supplier delivers them, and the restaurant can switch suppliers without changing how the chef cooks.

```
public interface IEmailSender { void Send(string to, string msg); }
public class SmtpEmailSender : IEmailSender { /* real email */ }
public class FakeEmailSender : IEmailSender { /* for tests  */ }

public class OrderService
{
    private readonly IEmailSender _email;
    public OrderService(IEmailSender email) => _email = email;   // injected
    public void Confirm(string user) => _email.Send(user, "Order confirmed");
}

// Registration (Program.cs): the container decides which class to inject
builder.Services.AddScoped<IEmailSender, SmtpEmailSender>();
```

Lifetime	Meaning	Example
Transient	New instance every time it is requested	Lightweight helper services
Scoped	One instance per HTTP request	DbContext, repositories
Singleton	One instance for the whole application	Cache, configuration

Q6. Difference between Web Pages (Razor) and MVC, two business cases, and a comparison

Aspect	Razor Pages	MVC
Organisation	Page-based: each page = .cshtml + PageModel (.cshtml.cs)	Controller + Model + View in separate folders
Routing	Based on file path (/Pages/Contact.cshtml gives /Contact)	Conventional or attribute routing to controller/action
Handling requests	Handlers inside the page: OnGet(), OnPost()	Action methods in a controller
Separation of concerns	Good, but logic is tied to its page	Strong, one controller can serve many views/APIs
Learning curve	Simpler, fewer files per feature	More structure and more concepts
Reuse	Reuse by partials / view components	One action/model can serve views, JSON, and APIs
Best for	Page-focused, form-heavy apps	Large, complex apps and APIs

Business case 1: Company website with contact and registration forms

A clinic site with Home, About, Services, "Book an appointment" and "Contact us" pages. Each page has its own simple form and little shared logic. **Razor Pages** fits best: one file pair per page, fast to build and easy for a small team to maintain.

Business case 2: E-commerce platform

Products, cart, orders, payments, an admin panel, a mobile app, and a public REST API all working on the same data. **MVC** fits best: controllers coordinate complex workflows, the same Order model can feed an HTML view, a JSON endpoint, and an admin view, and the clear separation makes testing and team work on a large codebase easier.

Comparison: Razor Pages gives faster development and less ceremony for page-centric scenarios; MVC gives better scalability and flexibility when many views/clients share logic. Both are part of ASP.NET Core and can be used in the same project.

Q7. What is Content-Type in the response message, where do we use it, and why?

Content-Type is an HTTP header that tells the receiver the **media type (MIME type)** of the body, and optionally its charset, so it knows how to read it. Example: `Content-Type: application/json; charset=utf-8`.

Content-Type	Body is	Typical use
text/html	HTML page	MVC views, Razor pages
application/json	JSON data	Web APIs, AJAX / fetch calls

Content-Type	Body is	Typical use
application/xml	XML data	Legacy services, SOAP
text/css, application/javascript	Stylesheet, script	Static files
image/png, image/jpeg	Image	Pictures, logos
application/pdf	PDF file	Invoices, reports downloads
multipart/form-data	Files + fields (request)	File upload forms
application/x-www-form-urlencoded	key=value (request)	Normal HTML form POST

Where we use it

- **Responses:** so the browser renders HTML, shows an image, or downloads a PDF.
- **Requests:** POST/PUT, so the server model binder knows whether to parse form data or JSON.
- **Content negotiation:** the client sends `Accept`, and the server answers with a matching Content-Type.

```
return View();                                  // text/html
return Json(new { id = 1 });                    // application/json
return File(bytes, "application/pdf");          // application/pdf
return Content("<h1>Hi</h1>", "text/html");     // custom type
```

Why it matters

- Without it the browser may display JSON as text, or show a PDF as garbage characters.
- **Security:** a wrong type lets browsers "sniff" and run uploaded content as a script (XSS); we reduce this with `X-Content-Type-Options: nosniff`.
- API clients (React, mobile apps) rely on it to parse the response correctly, and charset prevents broken Arabic/Unicode text.

Q8. What are minification, web bundling, Webpack, and lazy loading, and what is their role in network performance?

Technique	What it is	Effect on performance
Minification	Removing comments, spaces, line breaks and shortening variable names in JS/CSS/HTML without changing behavior	Smaller files, so less data to download
Bundling	Combining many JS/CSS files into one or a few files	Fewer HTTP requests, less connection overhead
Webpack	A module bundler for front-end apps: starts from an entry file, follows imports, and outputs optimized bundles. Uses loaders (Babel, CSS, images) and plugins (minify, hashing), and supports tree-shaking and code splitting	Automates minify + bundle + split + cache-busting in one build
Lazy loading	Loading a resource only when it is needed (a route, a component, an image below the fold) instead of at startup	Smaller first download, faster first paint / time to interactive

Example

```
// Before minification
function calculateTotal(price, qty) {
    // multiply price by quantity
    return price * qty;
```

```
}
// After minification
function calculateTotal(a,b){return a*b}
```

Webpack config sketch (entry, output, split, minify) and lazy loading in React:

```
// webpack.config.js
module.exports = {
  mode: 'production',                      // enables minification + tree-shaking
  entry: './src/index.js',
  output: { filename: '[name].[contenthash].js' },   // hash = better browser caching
  optimization: { splitChunks: { chunks: 'all' } }   // code splitting
};

// React lazy loading: Dashboard code downloads only when the route is opened
const Dashboard = React.lazy(() => import('./Dashboard'));
<Suspense fallback={<p>Loading...</p>}>
  <Dashboard />
</Suspense>

<!-- Lazy-load images -->
<img src="photo.jpg" loading="lazy" alt="photo">
```

Role in increasing performance through the network

- **Less data:** minification (and compression like gzip/Brotli) shrinks what travels over the network.
- **Fewer requests:** bundling reduces round trips and latency (this matters most on HTTP/1.1; on HTTP/2 the gain is smaller but still useful).
- **Load on demand:** lazy loading and code splitting postpone non-critical code, so the page becomes usable sooner.
- **Better caching:** hashed bundle names let the browser cache files for a long time and re-download only what changed.
- **Result:** faster load time, lower bandwidth cost, better user experience on slow mobile networks, and better SEO.